(19)中华人民共和国国家知识产权局

(12)发明专利申请

(10)申请公布号 CN 104765653 A
(43)申请公布日 2015.07.08

(21)申请号 201510083556.6

(22)申请日 2015.02.15

(71)申请人 杭州米加科技有限公司
地址 310013 浙江省杭州市西湖区紫荆花路2号联合大厦B座10楼

(72)发明人 周正中

(74)专利代理机构 杭州杭诚专利事务所有限公司 33109
代理人 尉伟敏

(51)Int.Cl.
G06F 11/14(2006.01)
G06F 17/30(2006.01)

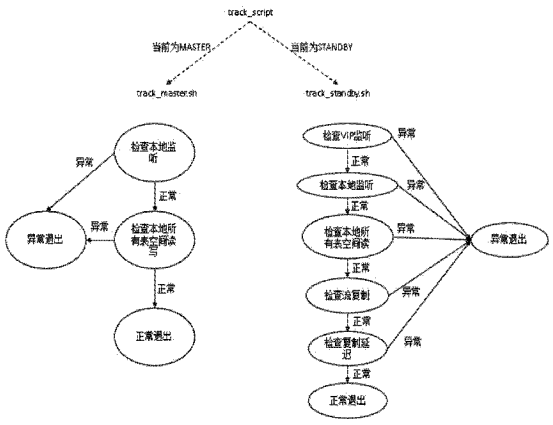
权利要求书1页 说明书4页 附图2页

(54)发明名称

一种使用keepalived软件实现数据库HA应用的方法

(57)摘要

本发明涉及一种使用keepalived软件实现数据库HA应用的方法，其特征在于，包括如下步骤：节点状态切换为master后，同步调用notify_*脚本，*是master、backup二者中的一个；等待notify_*脚本调用结束并返回结果为正常；触发跟踪脚本track_script。本发明通过在节点进入master状态后同步调用notify，避免了keepalived软件在数据库HA架构中遇到无休止切换的问题。此外，本发明通过设置软链接，实现了不同的状态下使用不同的track脚本。



CN 104765653 A

1.一种使用 keepalived 软件实现数据库 HA 应用的方法，其特征在于，包括如下步骤：

节点状态切换为 master 后，同步调用 notify_* 脚本，* 是 master、backup 二者中的一个；

等待 notify_* 脚本调用结束并返回结果为正常；

触发跟踪脚本 track_script。

2.根据权利要求 1 所述的一种使用 keepalived 软件实现数据库 HA 应用的方法，其特征在于，所述的 notify_mater 执行如下步骤：

fence 切换至 master 状态节点的对端节点；

激活 standby 数据库；

将 track_scrip.sh 文件软链接指向 track_master.sh 文件。

3.根据权利要求 2 所述的一种使用 keepalived 软件实现数据库 HA 应用的方法，其特征在于，所述的 track_master.sh 用于检查本地监听、检查本地表空间的读写中的至少一种。

4.根据权利要求 1 所述的一种使用 keepalived 软件实现数据库 HA 应用的方法，其特征在于，所述的 notify_backup 执行如下步骤：

关闭数据库；

将 track_scrip.sh 文件软链接指向 track_backup.sh 文件；

启动数据库。

5.根据权利要求 4 所述的一种使用 keepalived 软件实现数据库 HA 应用的方法，其特征在于，track_backup.sh 用于检查 VIP 监听、检查本地监听、检查本地表空间的读、检查流复制、检查复制延迟中的至少一种。

6.根据权利要求 1-5 中任意一条所述的一种使用 keepalived 软件实现数据库 HA 应用的方法，其特征在于，还包括如下步骤：

配置数据库流复制环境，备份数据库实时从主数据库获取 Xlog 信息并实时恢复；

启用 keepalived 主节点（master）、备节点（backup）。

一种使用 keepalived 软件实现数据库 HA 应用的方法

技术领域

[0001] 本发明涉及一种实现数据库 HA(高可用)应用的方法,尤其是一种使用 keepalived 软件实现数据库 HA 应用的方法。

背景技术

[0002] Keepalived 软件一般用于检测 Web 服务器的状态,如果有一台 web 服务器死机或工作出现故障,将被 keepalived 检测到,并将有故障的 web 服务器从系统中剔除,当 web 服务器工作正常后,keepalived 将自动将 web 服务器加入到服务器群中,上述这些工作全部自动完成,需要人介入参与的仅仅是修复故障的 web 服务器。

[0003] 除此之外,keepalived 还常常被用于实现基于 Linux 系统的应用负载均衡和高可用(HA),当 keepalived 服务启动时,主机角色(包括 master 、backup、fault)的选择一般遵循如图 1 所示的逻辑图。

[0004] 在设计数据库 HA 时, 非常重要的一点是确保不出现脑裂的情况(脑裂,是指主库和备库出现了不可逆转的差异数据)。为了防止脑裂, 数据库在进行主备切换时, 必须确保数据不会被多台主机修改, 或者必须确保 failded 主节点完全关机后才能激活备节点,为此,增加了 notify 和 fence 机制。具体来说,当一个节点进入 master 状态后,会触发 notify 和 fence 其对端服务器,激活数据库,但是,有可能出现在上述动作还没有完成前 track script 就被触发的情况,如果在 notify 动作完成前触发 track script 的话,会导致 failed 状态出现,进而将进入 master 状态的节点主机变成 fault 状态。从图 1 可以得出,当原 master 状态节点进入 fault 状态后,其他的 backup 状态节点则会开始竞选 master 资格,而竞选必将触发新的一轮 notify,那么,如果在随后每次触发 notify 时都出现在 notify 完成之前 track script 触发的情况,系统将永无休止的进行下去,而无法完成此次切换,如果多次出现上述情况,也必然导致系统切换时间成本的增加,效率降低。也就是说,在使用 Keepalived 来构建数据库的 HA 时,在 notify_* (* 是主机角色)和 notify script 调用结束前,如果 track (检查)发生了,那么可能会导致 track 结果为不正常,进而触发 master 角色又转变成 backup,而这次角色翻转又将再一次触发调用 notify_backup, notify script,如此循环往复(上述意外再次发生),则陷入永远无法完成切换的死循环。

发明内容

[0005] 为克服现有技术中存在的技术问题,本发明提出一种使用 keepalived 软件实现数据库 HA 应用的方法。

[0006] 为达到上述目的,本发明所采用的技术方案是 :

一种使用 keepalived 软件实现数据库 HA 应用的方法,其特征在于,包括如下步骤 :

节点状态切换为 master 后,同步调用 notify_* 脚本,* 是 master、backup 二者中的一个 ;等待 notify_* 脚本调用结束并返回结果为正常 ;触发跟踪脚本 track_script。

[0007] 上述技术方案还可以进一步完善 :

作为优选，所述的 notify_mater 执行如下步骤：fence 切换至 master 状态节点的对端节点；激活 standby 数据库；将 track_scrip.sh 文件软链接指向 track_master.sh 文件。

[0008] 作为优选，所述的 track_master.sh 用于检查本地监听、检查本地表空间的读写中的至少一种。

[0009] 作为优选，所述的 notify_backup 执行如下步骤：关闭数据库；将 track_scrip.sh 文件软链接指向 track_backup.sh 文件；启动数据库。

[0010] 作为优选，track_backup.sh 用于检查 VIP 监听、检查本地监听、检查本地表空间的读、检查流复制、检查复制延迟中的至少一种。

[0011] 作为优选，使用 keepalived 软件实现数据库 HA 应用的方法还包括如下步骤：配置数据库流复制环境，备份数据库实时从主数据库获取 Xlog 信息并实时恢复；启用 keepalived 主节点(master)、备节点(backup)。

[0012] 由于上述技术方案的采用，本发明具有以下优点：

本发明通过在节点进入 master 状态后同步调用 notify，避免了 keepalived 软件在数据库 HA 架构中遇到无休止切换的问题。此外，本发明通过设置软链接，实现了不同的状态下使用不同的 track 脚本。

附图说明

[0013] 图 1 是 keepalived 服务启动时，各主机角色的选择逻辑图；

图 2 是一种数据库跟踪脚本逻辑图。

具体实施方式

[0014] 下面通过实施例并结合附图，对本发明的技术方案做进一步解释说明。

[0015] Keepalived 一般包含 3 个组件，分别是核心组件，监控检查组件以及 VRRP 组件，其中：

1、核心组件用于解析配置文件，FORK 监控和 VRRP 子进程，监控子进程的状态并在必要的时候重启子进程。

[0016] 2、监控检查组件负责监控负载均衡主机的状态。

[0017] 3、VRRP 组件遵循 VRRP 协议设计，用于检测主机状态，接口状态，负责主机角色的转换，主机角色包含 MA STER, BACKUP, FAULT 三种。

[0018] 在配置 keepalived 前，需要对数据库 HA 环境进行系统规划，如配置至少 2 台主机，与主机数量对应的 IP，并另外设置至少一个虚拟 IP 用于漂移(跟随角色转换进行漂移，master 在哪台主机就漂移到哪台物理主机上，因此，应用程序只需要链接到虚拟 IP 即可，而无需关注 master 库在哪个物理主机上)。配置 keepalived 时，将各主机 IP 加入单播 peer，以确定 keepalived 发送和接收 vrrp 包的范围。

[0019] 以下介绍一种实施例，应用本实施例，可以使 keepalived 用于构建数据库 HA，并应用于大多数的数据库 HA 架构中，如基于流复制的 HA、基于块设备复制的 HA、基于共享存储的 HA、基于分布式存储的 HA。

[0020] 一种使用 keepalived 软件实现数据库 HA 应用的方法，包括如下步骤：

(1) 节点状态切换为 master 后，同步调用 notify_* 脚本，* 是 master、backup 二者中

的一个；

（2）等待 notify_* 脚本调用结束并返回结果为正常；

（3）触发跟踪脚本 track_script；

（4）配置数据库流复制环境，备份数据库使用流复制协议，实时从主数据库获取 Xlog 信息并实时恢复；

（5）启用 keepalived 主节点（master）、备节点（backup）。

[0021] 一种实施例中，通过使用主节点、备节点共同的基线，以避免脑裂发生。

[0022] 上述步骤（1）中，如调用 notify_mater，则 notify_mater 执行如下步骤：fence 切换至 master 状态节点的对端节点；激活 standby 数据库；将 track_scrip.sh 文件软链接指向 track_master.sh 文件。

[0023] 图 2 所示为一种跟踪脚本 track_master.sh 的检查示例，track_master.sh 用于检查本地监听、检查本地表空间的读写，在一个实施例中，返回检查结果为 0 表示正常，返回结果非 0 表示不正常。在更多的实施例中，检查项目顺序不限于图 2 所示。

[0024] 上述步骤（1）中，如调用 notify_backup，则 notify_backup 执行如下步骤：关闭数据库；将 recovery.done 改成 recovery.conf；将 pg_xlog 重命名，并新建一个 pg_xlog，对其赋予 700 权限（文件用户可读、写、运行，同组用户和其他用户不可执行任何操作）；将 track_scrip.sh 文件软链接指向 track_backup.sh 文件；启动数据库。

[0025] 图 2 所示为一种跟踪脚本 track_backup.sh 的检查示例，track_backup.sh 用于检查 VIP 监听、检查本地监听、检查本地表空间的读、检查流复制、检查复制延迟，在一个实施例中，返回检查结果为 0 表示正常，返回结果非 0 表示不正常。在更多的实施例中，检查项目顺序不限于图 2 所示。

[0026] 本领域内的技术人员应明白，本发明的实施例可提供为方法、系统、或计算机程序产品。因此，本发明可采用完全硬件实施例、完全软件实施例、或结合软件和硬件方面的实施例的形式。而且，本发明可采用在一个或多个其中包含有计算机可用程序代码的计算机可用存储介质（包括但不限于磁盘存储器、CD-ROM、光学存储器等）上实施的计算机程序产品的形式。

[0027] 本发明是参照根据本发明实施例的方法、设备（系统）、和计算机程序产品的流程图和 / 或方框图来描述的。应理解可由计算机程序指令实现流程图和 / 或方框图中的每一流程和 / 或方框、以及流程图和 / 或方框图中的流程和 / 或方框的结合。可提供这些计算机程序指令到通用计算机、专用计算机、嵌入式处理机或其他可编程数据处理设备的处理器以产生一个机器，使得通过计算机或其他可编程数据处理设备的处理器执行的指令产生用于实现在流程图一个流程或多个流程和 / 或方框图一个方框或多个方框中指定的功能的装置。

[0028] 这些计算机程序指令也可存储在能引导计算机或其他可编程数据处理设备以特定方式工作的计算机可读存储器中，使得存储在该计算机可读存储器中的指令产生包括指令装置的制造品，该指令装置实现在流程图一个流程或多个流程和 / 或方框图一个方框或多个方框中指定的功能。

[0029] 这些计算机程序指令也可装载到计算机或其他可编程数据处理设备上，使得在计算机或其他可编程设备上执行一系列操作步骤以产生计算机实现的处理，从而在计算机或

其他可编程设备上执行的指令提供用于实现在流程图一个流程或多个流程和 / 或方框图一个方框或多个方框中指定的功能的步骤。

[0030] 尽管已描述了本发明的优选实施例，但本领域内的技术人员一旦得知了基本创造性概念，则可对这些实施例作出另外的变更和修改。所以，所附权利要求意欲解释为包括优选实施例以及落入本发明范围的所有变更和修改。

[0031] 显然，本领域的技术人员可以对本发明进行各种改动和变型而不脱离本发明的精神和范围。这样，倘若本发明的这些修改和变型属于本发明权利要求及其等同技术的范围之内，则本发明也意图包含这些改动和变型在内。

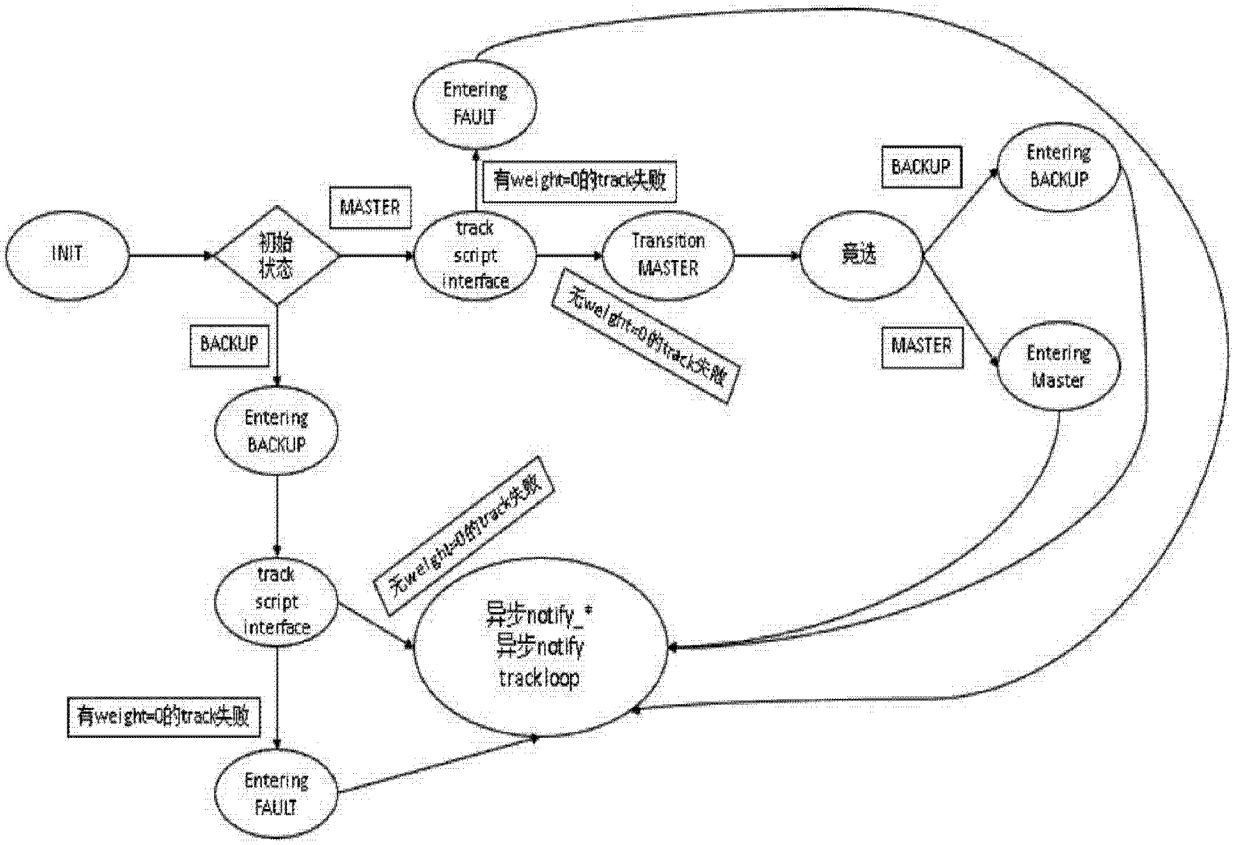

Entering FAULT
有weight=0的track失败
MASTER
INIT
初始状态
track script interface
Transition MASTER
竞选
BACKUP
Entering BACKUP
无weight=0的track失败
BACKUP
Entering BACKUP
MASTER
Entering Master
track script interface
无weight=0的track失败
异步notify_* 异步notify trackloop
有weight=0的track失败
Entering FAULT

图 1

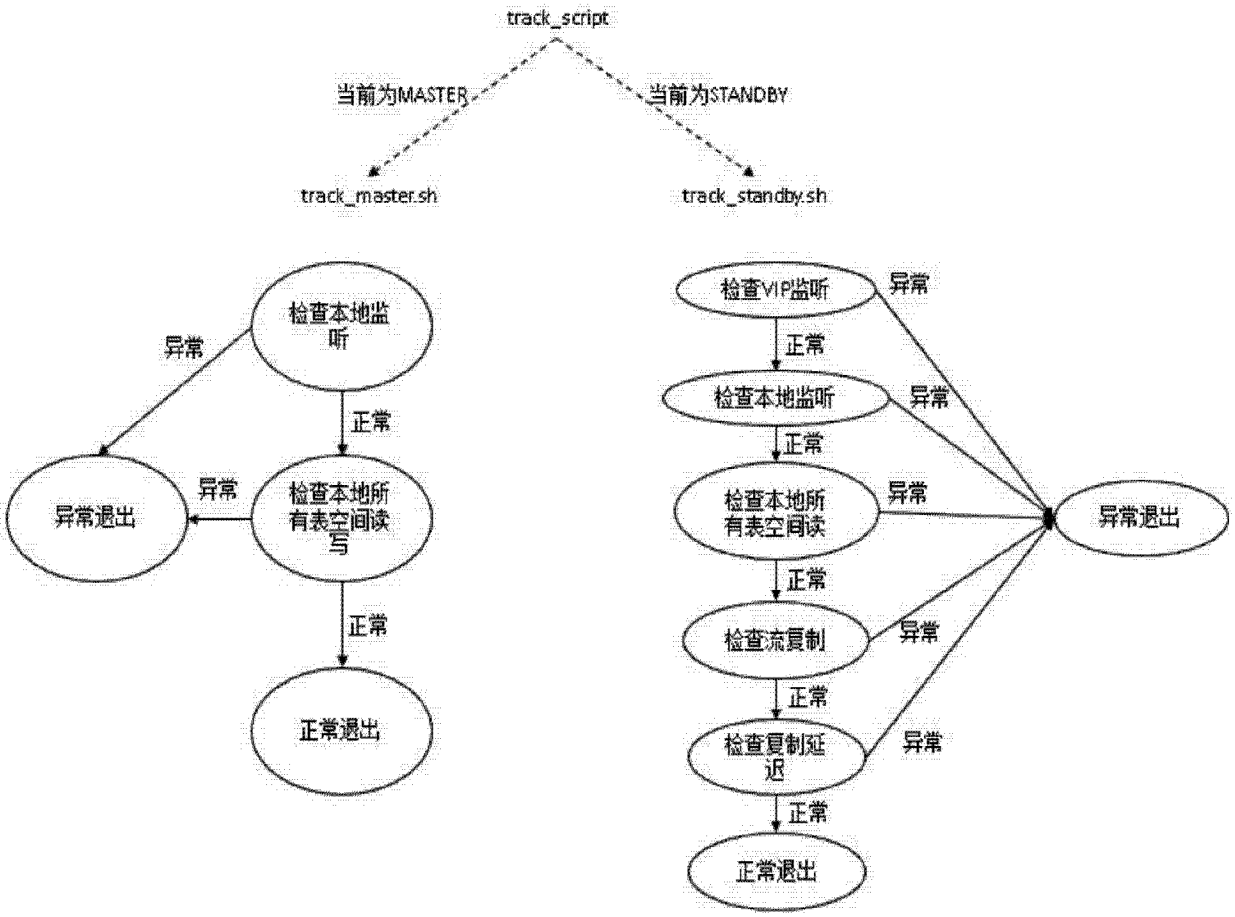

track_script
当前为MASTER
当前为STANDBY
track_master.sh
track_standby.sh
检查本地监听
异常
正常
异常退出
检查本地所有表空间读写
正常退出
检查VIP监听
检查本地监听
检查本地所有表空间读
检查流复制
检查复制延迟
异常退出

图 2